

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

ASSIGNMENT 2

Please submit the answers to the following questions in Brightspace by the due time indicated in the submission entry. There are 20 points in total + 6 bonus points (weight: 0.25).

Submit your answers as a **gzipped tarball** "username-comp3000-assign2.tar.gz" (where username is your MyCarletonOne username). Do NOT just submit a file of another format (e.g., txt or docx) by renaming it to .tar.gz. Unlike tutorials, assignments are graded for the correctness of the answers.

The tarball you submit **must** contain the following (applicable to all assignments):

1. A **plaintext** file containing your solutions to all questions, including explanations.
2. A README.txt file listing the contents of your submission as well as any information the TAs should know when grading your assignment. Without this file, grading will be based on the TA's understanding.
3. For each question, where applicable, a C file for your modified version of the original file. This should include all required changes for that question. This way, you can avoid something wrong with one question affecting another question.
4. Diff files showing the modifications, by comparing each submitted C file above and the original: for example, `diff -c 3000original.c 3000modified.c > 3000.diff`. Avoid moving around or changing existing code (unless necessary) which may be distracting.

For this assignment, you need to submit code for questions 1, 2, 4, and 5 (if you choose to answer it) in part 1. There is no code to be submitted for part 2.

You can use this command to create the tarball: `tar zcvf username-comp3000-assign2.tar.gz your_assignment_directory`. ****Don't forget to include your plaintext file!!****

No other formats will be accepted. Submitting in another format will likely result in your assignment not being graded and you receiving no marks for this assignment. In particular, do not submit an MS Word, OpenOffice, or PDF file as your answers document!

As a student learning operating system, you are expected to be able to handle file transfer properly using necessary commands such as `ssh`, `scp` or `rsync`. Otherwise (which happens), **please set aside time by completing your assignment enough long before the due time**, then you may ask a TA, the instructor, or another student for any file transfer issue. Then, you can keep working on it and **override your submission** by uploading a new tarball before it is due.

For your convenience, [here are a few commands](#) (among many others) to download a file from your Openstack VM. You need to run the command on your PC, not your Openstack VM.

Empty or corrupted tarballs may be given a grade of zero, so please **double check your submission by downloading and extracting it**.

Don't forget to include what outside resources you used to complete each of your answers, including other students, and web resources. You do not need to list help from the instructor, TA, or information found in the textbook or man pages.

Use of any outside resources verbatim as your answer (like copy-paste or quotation) is not allowed, and will be treated as unauthorized collaboration (and reported as plagiarism).

Please do **NOT** post assignment solutions on MS Teams or Brightspace (or other platforms not used in the course such as [Discord](#)) or it will be penalized. Moreover, [posting the assignment questions to any external forums/websites is NOT permitted and will also be penalized/reported](#).

Questions – part 1 [13 + BONUS]

The following questions (where applicable) will be based on the original [3000shell.c](#) in Tutorial 3. All unmentioned functionalities of 3000shell should remain the same.

1. [4] Implement “appending” support for standard output redirection, with the operator “>>”. This would be similar to the already implemented standard output redirection. Then, you will be able to “command >>output.txt”. For example:

```
$ cat output.txt
a
$ echo -n b >>output.txt
$ cat output.txt
ab
```

It must work even if output.txt does not exist yet.

2. [4] Bash supports customizable command prompts with variables like PS0/PS1 (see `~/.bashrc`). We can also specify a string (not containing spaces or escape characters) as the custom prompt when invoking 3000shell. Modify the 3000shell code to achieve this:

```
# MYPROMPT=?? ./3000shell
student (23:11:48) ??
student (23:11:49) ??
```

Now the original “\$” has been replaced by “??”, preceded by the current system time in parentheses, in the exact format shown above (HH:MM:SS 24hrs). The time must be up to date for each new prompt (e.g., when hitting Enter).

3. [2] Replace the exec function (line 245) with “execv(bin_fn, args)”.

```
# export MYVAR=somevalue
# ./3000shell
student $ printenv MYVAR
somevalue
```

You should be able to see your MYVAR as the output all along, before and after the change.

Briefly explain why (but be specific).

(no code submission for this question)

4. [3] 3000shell is missing some common internal (built-in) commands, such as `cd` and `export`. Next, let's implement (sort of) the opposite of `export`: `unset`.

```
# export MYVAR=somevalue
# ./3000shell
student $ printenv MYVAR
somevalue
student $ unset MYVAR
student $ printenv MYVAR
student $
```

Note that after unset, you should not even see a blank line for MYVAR.

5. [6 BONUS] We have explained why appending the `&` operator to a command in 3000shell renders the terminal unresponsive for interactive programs (due to both processes vying for the shared

terminal). In this optional task, let's make 3000shell do it right. You can simply check how "&" behaves in bash, i.e., interactive programs run in the background will be suspended (stopped) and the shell still remains functional. You can also see this stopped program using the "jobs" command in bash. With the command "fg", you can bring it back to foreground.

What is happening behind the scene is: when a program is switched to background, attempts to read from `stdin` or write to `stdout` will trigger `SIGTTIN/SIGTTOU`, stopping the program.

Task: use one or multiple of the `setpgid()/getpgid()`, `tcsetpgrp()/tcgetpgrp()` functions to switch the program to background.

- Now running a program in the background (with `&`) will not lead to unresponsiveness. For non-interactive programs, the command just runs and finishes normally, and returns to the prompt (e.g., `ls`, see below). For interactive programs, it just returns to the prompt directly (e.g., `htop`, `vi` or `nano`, see below).
Using "ps aux", you should see in the STAT column a "T" indicating the program is in a stopped state.
- Afterwards, the shell must remain usable (taking and running new commands) instead of getting stuck.
- The expected changes will only be a few lines of code.
- You may want to add the `SA_NOCLDSTOP` flag (since `SIGCHLD` is also triggered when the a child process is stopped) with signal registration, and change from `wait()` to `waitpid()` in the signal handler.
- No need to consider switching it back to the foreground unless you are interested.

```
student $ ls &

Process 135678 running in the background.

student $ file1.txt file2.c file3 file4.so file5.c

Process 0 exited with status 0.

student $

student $ vi &

Process 138988 running in the background.

student $
```

Questions – part 2 [7]

The following questions (where applicable) will be based on the **patched** version of [3000userlogin.c](#) in Tutorial 4. No code submission for this part.

- [2] You will notice the patched version does not compile and produces an error (`undefined reference`). What does this error mean [1/2]? Try to explain using the knowledge related to static/dynamic linking and be specific to the code change introduced by the patched version. What specific option can you add to `gcc` to get this program compiled [1/2]?
- [3] In Tutorial 4, if you compile the code **without configuring `setuid root`**, you can at least log in with your current username (e.g., `student`). However, for this patched 3000userlogin, without `setuid root`, you cannot log in as any user, always with the error message "`Could not find user`". Which line of code is the root cause of this difference (mention the line number or paste the line as

your answer) [1/3]? Using which command can clearly tell you this [1/3] (there could be multiple possible answers) with some brief explanation [1/3]?

8. [2] What is the purpose of the call to `memcpy()` (line 39) [1/2]? Put another way, what will happen if this operation is not performed?
Why is line 47 necessary [1/2]? You need to explain what it actually does (not what error you see without it).
9. Not a question and no bonus: you can see the typed password is being displayed in cleartext to the terminal which is not good for privacy (unlike the password prompt of the SSH login). When you gradually get familiar with the terminal you will be able to turn off “echo” using `termios`.